World Sensing Music Station Design Document

A handheld instrument that turns movement through space into layered music.

1) Core Idea

The World Sensing Loop Music Station is an interactive music device that treats the physical world like an "instrument." Instead of playing notes with a traditional keyboard, the user moves through space and the device senses distance and motion using an ultrasonic sensor. That sensing becomes music in real time—forming short repeating loops that evolve based on where you stand, how fast you move, and how consistently you repeat routes.

The concept is inspired by the idea that daily movement patterns (walking from the door to the desk, hallway to bedroom, etc.) are a form of rhythm. The device captures those spatial patterns and turns them into musical patterns—so the environment "plays itself."

2) Design Goals

1. Make music from movement, not from technical music knowledge
   The user should be able to make something musical quickly without understanding theory.
2. Keep interaction simple and physical
   A few buttons, clear feedback, minimal menus.
3. Make results feel "loop-based," not random noise
   Loops should have structure (beats repeat, layers stack).
4. Show state clearly
   Users must always know: mode, which layer is selected, and whether it's recording or playing.

3) System Architecture (Two-Board Structure)

This project intentionally uses a two-board Arduino structure to separate responsibilities and keep the system modular:

Board A: "Control" (Brain / Sensing / Audio / LED Strip)

- Reads ultrasonic distance (TRIG/ECHO)
- Generates music logic (sequencer + rules)
- Outputs sound via TRRS audio jack
- Drives NeoPixel LED strip for visual feedback
- Reads 7 buttons (UP/LEFT/DOWN/RIGHT, and 3 modes)
- Sends button commands, text and status messages to the display board over Serial communication

Board B: "Display" (UI Input + Matrix Display)

- Displays mode/status text on the LED matrix (scrolling text)

This split is useful because timing-heavy LED matrix scanning and musical timing can conflict. Separating display and control makes it easier to expand later.

4) Musical Interaction Design

4.1 3 Musical Modes (Iconic, distinct)

- AMBIENT: sparse, cinematic, slow, spacey scale (pentatonic feel)
- TRAP: modern groove, half-time snare anchor, busier hats
- TECHNO: 4-on-the-floor kick feel, consistent energy

Modes act like "stylistic presets." They change tempo, swing, rhythmic rules, and the scale/motif mapping.

4.2 3 Layers (Three Loops Stacked)

The device builds music by stacking three 8-step loops:

1. Rhythm loop (kick/snare/hat)
2. Bass loop
3. Melody loop

This makes output feel structured and "produced" instead of a single-line beep.

4.3 Replace-One-Loop Interaction

A key design decision: the user should be able to change part of the music without destroying everything.

So instead of constantly overwriting the loop, the instrument uses:

- PLAY mode by default (the loops stay stable)
- REPLACE mode for exactly one full loop cycle (8 steps) when triggered
  After 8 steps, it automatically returns to PLAY.

That gives the user a clean mental model:

- "I like my rhythm loop—let me replace only the melody loop."

5) How to Use It (User Instructions)

Hardware Controls

- Mode buttons (Left to Right):
  - AMBIENT
  - TRAP
  - TECHNO
- Directional buttons
  - LEFT: cycle selected layer (Rhythm → Bass → Melody)
  - DOWN: replace selected layer for one loop (records for 8 steps, then stops)
  - UP: clear all loops (full reset)
  - RIGHT: randomize the selected layer (quick inspiration)

What the LEDs mean

- NeoPixel LED 0/1/2 = which layer is selected
- Remaining LEDs visualize the 8 steps and activity
- LED matrix shows mode + bpm + layer + REC/PLAY status

How to perform with it (simple workflow)

1. Pick a mode (Ambient/Trap/Techno)
2. Select Rhythm layer → press DOWN → move in space for 8 beats
3. Select Bass layer → press DOWN → move differently for 8 beats
4. Select Melody layer → press DOWN → make more expressive movements
   Now you have a full 3-layer loop.

6) How I Realized It (Implementation Notes)

6.1 Distance → Musical Decisions

The ultrasonic sensor gives distance in cm. I process it with:

- range clamp (ignore invalid values)
- smoothing filter (exponential moving average) to reduce jitter
- zone mapping: map distance into 8 "zones" (0–7)
- speed proxy: use change in smoothed distance as "movement speed"

Then I apply rule-based mapping:

- Rhythm: kick/snare/hat patterns depend on mode + step index + speed
- Bass/Melody: zone selects a note index in a constrained scale/motif

6.2 Making It Sound Musical Without Full Synthesis

Because Arduino Nano audio is limited, I use:

- tone() for frequencies

- "time-slicing" inside each step so you hear:
    - short drum click portion
    - bass note portion
    - melody note portion

This mimics layered tracks even though the output is single-channel.

6.3 Visual Feedback (NeoPixel + Matrix)

- NeoPixel strip shows state and loop activity continuously
- Matrix display is for clear text status ("AMBIENT 80bpm MELO PLAY")
  This helps users understand what's happening instantly.

7) Potential User Scenarios & Use Cases

7.1 Walking Route Music (Daily Ritual)

A student walks the same route every day (dorm → classroom).
The instrument turns that habitual route into a repeatable loop, creating a personal "theme song" for their routine.

7.2 Performance / Live Instrument

A performer uses it on stage, moving their hands or body to "record" new loops layer by layer while the previous layers keep playing.

7.3 Interactive Installation

Mounted in a hallway or gallery space:

- distance from viewers affects melody/bass
- foot traffic creates rhythm density
  The space becomes a living music machine.

7.4 Creative Tool for Non-Musicians

People who don't play instruments can still create structured loops by moving. It's a music creation interface based on body movement, not technique.

8) Limitations

- tone() audio is still "beepy" and lacks real drum timbre.
- Ultrasonic sensors can be noisy in reflective or crowded environments.
- Current loop length is fixed (8 steps), and layers are time-sliced rather than truly polyphonic.

- Without a screen, explaining controls requires good labeling or documentation.

9) Improvements & Future Work

Audio Quality

- Replace tone() with simple synthesis:
    - envelope (attack/decay) so notes feel less harsh
    - noise-based hi-hats (PWM noise)
    - simple kick drum pitch drop (classic drum synth trick)

More Musical Structure

- Allow longer loops (8 / 16 / 32 steps)
- Add "variation every 4 loops" (fills, bass movement, melody change)
- Add probability-based rhythm so it evolves but stays coherent

Better Sensing

- Add a second ultrasonic sensor facing another direction (2D sensing)
- Or switch to IMU (accelerometer/gyro) for smoother "motion-based" control

Better UI

- Label buttons physically
- Add a small OLED or use matrix icons for clearer state
- Add "count-in" when replacing a loop (beep + LED countdown)

Recording/Performance Features

- Add mute/solo per layer
- Add tempo tap or tempo slider
- Add memory (save last loops to EEPROM)